

API Architecture & Integration Documentation

Version 2.0.0 | Generated 2026-10-02 14:36:21

1. System Overview

Trevolk Forecasting Engine is a production-grade FastAPI microservice that serves a trained **XGBoost regression model** for 7-day Rossmann store-sales prediction. It is consumed by a Next.js frontend dashboard that renders KPI cards and a daily forecast table in real-time.

Key characteristics:

- Modular package layout (app/main, app/api, app/schemas, app/services)
- Thread-safe singleton MLService — model loaded once at startup
- Batch inference across a 7-day rolling window
- Strict Pydantic v2 request/response validation
- CORS pre-configured for Next.js (localhost:3000)
- OpenAPI / Swagger UI available at /docs

2. Directory Structure

```
Trevolk_Forecasting_Engine/
├── app/
│   ├── __init__.py
│   ├── main.py           ← FastAPI app, CORS, lifespan, routers
│   └── api/
│       ├── __init__.py
│       ├── endpoints.py  ← POST /api/predict route
│       └── schemas/
│           ├── __init__.py
│           └── predict.py ← ForecastRequest, ForecastResponse
│   └── services/
│       ├── __init__.py
│       └── ml_service.py  ← Singleton, feature engineering, inference
├── trevolk_sales_model.pkl ← Trained XGBoost model (required)
├── generate_pdf_docs.py    ← This documentation generator
├── requirements.txt
└── Trevolk_API_Architecture.pdf ← Generated output
```

Layer Responsibilities

Layer	File	Responsibility
Entry Point	app/main.py	App init, CORS, lifespan model warm-up, router registration
Router	app/api/endpoints.py	HTTP routing, error mapping to status codes
Schema	app/schemas/predict.py	Pydantic validation for request and response
Service	app/services/ml_service.py	Model loading, feature engineering, batch inference
Script	generate_pdf_docs.py	Standalone PDF documentation generator

3. API Endpoints

Method	Path	Description	Auth
GET	/	Service root — returns version and status	None
GET	/health	Readiness probe — confirms model is loaded	None
POST	/api/predict	7-day sales forecast	None
GET	/docs	Interactive Swagger UI (OpenAPI)	None
GET	/redoc	ReDoc API documentation	None

3.1 POST /api/predict

Accepts a JSON payload from the Next.js frontend, builds a 7×12 feature matrix, runs a single batch XGBoost inference call, and returns structured KPI data.

4. Payload Structures

4.1 Request Payload (ForecastRequest)

```
{
  "store_id":          1,
  "forecast_start_date": "2026-10-02",
  "promotion_active":   true,
  "school_holiday":     false,
  "competition_distance": 1270.0
}
```

Field	Type	Constraints	Description
store_id	integer	>= 1	Rossmann store identifier
forecast_start_date	string	YYYY-MM-DD, real date	First day of 7-day window
promotion_active	boolean	—	Whether a promo is running
school_holiday	boolean	—	Whether schools are closed
competition_distance	float	>= 0.0	Metres to nearest competitor

4.2 Response Payload (ForecastResponse)

```
{
  "expected_revenue":      57893.5,
  "optimal_price_adjustment": 4.8,
  "confidence_score":      95.0,
  "daily_forecast": [
    { "date": "2026-10-02", "day": "Fri", "predicted_sales": 8257.0 },
    { "date": "2026-10-03", "day": "Sat", "predicted_sales": 9412.0 },
    ...
  ]
}
```

Field	Type	Description
expected_revenue	float	Sum of all 7 daily predictions — drives the Revenue KPI card

Field	Type	Description
optimal_price_adjustment	float	CV-based price signal in percentage points (± 10 % band)
confidence_score	float (0–100)	Model accuracy indicator — currently static at 95.0
daily_forecast	array	7 objects: {date, day, predicted_sales} — drives the table

5. ML Model Integration

5.1 Model File

The trained XGBoost regressor is serialised with **joblib** and stored as **trevolk_sales_model.pkl** in the project root. It must be present before the server starts.

5.2 Feature Engineering

For each incoming request, MLService generates 7 rows — one per forecast day — by decomposing *forecast_start_date* and the following 6 dates:

Feature	Source	Notes
Store	store_id (request)	Direct mapping
DayOfWeek	forecast_start_date + offset	Rossmann convention: 1=Mon ... 7=Sun
Promo	promotion_active (request)	bool $\rightarrow$ int (0/1)
SchoolHoliday	school_holiday (request)	bool $\rightarrow$ int (0/1)
StoreType	DEFAULT_STORE_TYPE = 0	Mocked — replace with label-encoded value
Assortment	DEFAULT_ASSORTMENT = 0	Mocked — replace with label-encoded value
CompetitionDistance	competition_distance (request)	Float, metres
Year	forecast_start_date + offset	4-digit year
Month	forecast_start_date + offset	1–12
Day	forecast_start_date + offset	1–31
WeekOfYear	forecast_start_date + offset	ISO week number
StateHoliday	DEFAULT_STATE_HOLIDAY = 0	Mocked — replace with real encoding

5.3 Inference Pipeline

1. Singleton MLService.load_model() deserialises the .pkl file at startup.
2. ForecastRequest is validated by Pydantic before reaching the service.
3. _build_feature_matrix() produces a (7 $\times$ 12) pd.DataFrame in FEATURE_ORDER.
4. model.predict(df) runs a single vectorised XGBoost batch inference.
5. Predictions are clipped to ≥ 0 (sales cannot be negative).
6. KPIs (expected_revenue, optimal_price_adjustment) are computed from the array.
7. ForecastResponse is serialised and returned as JSON.

6. Operations Guide

6.1 Local Development

```
# Install dependencies
pip install -r requirements.txt

# Start the API server (auto-reload)
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# Open interactive API docs
http://localhost:8000/docs
```

6.2 Environment Variables

Variable	Default	Description
MODEL_PATH	trevolk_sales_model.pkl	Relative/absolute path to the .pkl file
PORT	8000	Server listen port
LOG_LEVEL	info	Uvicorn / Python logging level

6.3 Health Check

The **GET /health** endpoint returns HTTP 200 with `model_loaded: true` when the service is ready. Use this for Kubernetes liveness/readiness probes or load-balancer health checks.

7. Error Reference

HTTP Code	Scenario	Response body
200 OK	Successful forecast	{"expected_revenue": ..., "daily_forecast": [...]}
422 Unprocessable Entity	Invalid payload (Pydantic validation)	{"detail": [{"loc": ..., "msg": ...}]}
503 Service Unavailable	Model not yet loaded	{"detail": "Model is not available ..."}
500 Internal Server Error	Unexpected inference crash	{"detail": "Inference error: ..."}